

Attention, Transformers & Language Modeling

ECE 247B: DEEP
LEARNING II

Shreyas Rajesh

UCLA ECE
Spring 2026

From generative modeling of text
to GPT-style next-token
pretraining.

Why Study LLMs Now?

TRAINING COST

\$670 -> \$100M+

Frontier training went from the original Transformer scale to GPT-4/Gemini Ultra estimates in the tens to hundreds of millions. Epoch projects >\$1B runs by 2027.

A statistical objective became an industrial capital project.

INFRASTRUCTURE

\$660-690B

Microsoft, Alphabet, Amazon, Meta, and Oracle are estimated to commit this much capex in 2026, mostly to AI infrastructure and data centers.

A few firms are spending at national-infrastructure scale.

ADOPTION

900M weekly

ChatGPT reportedly reached 900M weekly active users in February 2026, after becoming the fastest consumer app to 100M users in early 2023.

Language models moved from lab artifact to daily interface.

TOKEN

THROUGHPUT

trillions/day

One inference provider reports 15T tokens/day; ChatGPT processes billions of prompts/day. The unit of work is now the token.

Scale is not just training: deployment is continuous.

POWER

50 GW by 2028

Anthropic estimates the U.S. AI sector may need at least 50 GW of electric capacity by 2028, with 2-5 GW data centers for frontier model development.

The bottleneck is becoming physical infrastructure.

In under a decade, language modeling became a story about probability, data, compute, users, and power. Today we unpack the core machine learning object at the center of it.

Sources/estimates: Epoch AI 2024/2026; Futurum Group 2026; Dell'Oro/TrendForce 2026; OpenAI/TechCrunch 2026; Fireworks AI/Business Insider 2026; Anthropic 2025. Figures vary by methodology.

The LLM Block: One Story in Three Lectures

LECTURE 1

From prediction to scaling laws

Shannon's prediction problem; next-token pretraining; attention and decoder-only transformers; GPT-2/3, scaling laws, and Chinchilla.

LECTURE 2

From scale to useful assistants

Why base models are not chatbots; instruction tuning, RLHF/DPO, ChatGPT, and modern deployment constraints.

LECTURE 3

Reasoning models and beyond

RAG, tools, agents, o-style models, DeepSeek-R1, and test-time compute as the next scaling axis.

Project connection: Project 3 is a compact version of today's story: a bigram baseline, a small GPT-style transformer on TinyStories, and generation comparison.

The Problem of Interest

Generative modeling of text

Sample

Draw new text from the learned distribution.

$$x \sim p_\theta$$

Evaluate likelihood

Assign probability to observed text.

$$\log p_\theta(x)$$

We observe text sequences $x = (x_1, \dots, x_T)$.

The goal is to learn a distribution close to $p_{\text{data}}(x)$.

Why text gives us a special route

Tokens are discrete.

Order is part of the data.

Each prefix gives a target: the next token.

The

model

predicts

?

Today is about turning this observation into an objective, an architecture, and eventually the GPT scaling recipe.

Aside: Entropy Measures Uncertainty

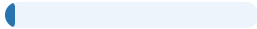
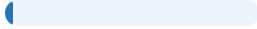
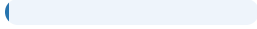
For one next-token distribution

$$H(p) = - \sum_i p_i \log_2 p_i$$

- Low entropy: probability mass is concentrated.
- High entropy: many tokens remain plausible.
- Cross-entropy asks how surprised our model is by the actual next token.

Low entropy

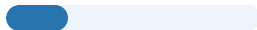
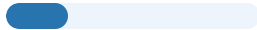
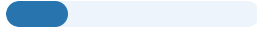
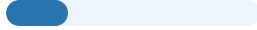
After the prefix **The cat sat on the**, one continuation may dominate.

mat		0.92
floor		0.04
rug		0.03
moon		0.01

H = 0.51 bits

High entropy

After the prefix **In this lecture we discuss**, several continuations may remain plausible.

attention		0.25
scaling		0.25
training		0.25
data		0.25

H = 2.00 bits

Language modeling is a game of reducing uncertainty: use the prefix to put probability mass on the next token that actually appears.

Generative Modeling Recipes We Have Seen

We have already studied several ways to learn a distribution and sample from it. The question now is which recipe fits ordered, discrete text.

GANs

Learn to generate samples by competing with a discriminator. Strong samples, but no direct likelihood.

VAEs

Introduce latent variables and optimize an evidence lower bound. Likelihood-based, but approximate.

Diffusion

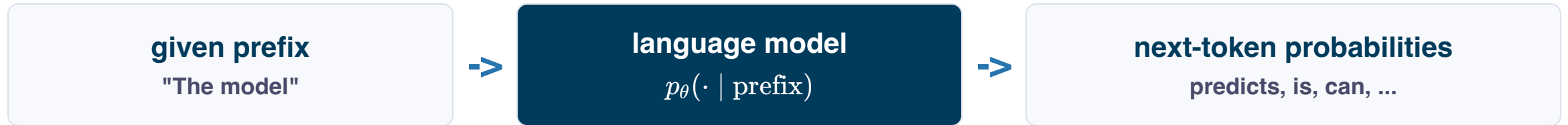
Corrupt data and learn to denoise. Extremely effective for continuous signals such as images.

Flow matching

Learn a velocity field that transports a simple distribution into the data distribution.

Today we add one more recipe: train a model to answer: given this prefix, what token comes next?

The Simple Route: Autoregression



Sampling

Start with a prefix, sample the next token, append it to the prefix, and repeat.

Likelihood evaluation

For real text, score each observed token under the distribution predicted from its prefix.

$$p_{\theta}(x_{1:T}) = \prod_{t=1}^T p_{\theta}(x_t \mid x_{<t})$$

The chain rule gives an exact density factorization. Learning becomes many supervised next-token classification problems.

Today's Route

Target for the end of lecture: Given a sequence of token ids, explain how a GPT-style model turns it into a distribution over the next token.

01

Problem

Frame text generation as learning a probability distribution.

02

Objective

Derive autoregressive likelihood, cross-entropy, and perplexity.

03

Context

See why long-range dependencies break simple count models.

04

Architecture

Use attention and decoder blocks to build GPT-style models.

05

Scaling

GPT-2/3 and Chinchilla show what to scale: parameters, compute, and data.

The Language Modeling Problem

An Old Problem

A Mathematical Theory of Communication

C. E. Shannon, Bell System Technical Journal, 1948.

The core question: Given a sequence of symbols, what comes next?

In 1948, Shannon asked whether we can model the statistical structure of English.

His approach:

- Count how often sequences of characters or words appear together.
- Use those statistics to sample new text.
- Increase context order and watch the samples become more language-like.

Project 3 starts with this same idea in the bigram baseline, then replaces the count table with a transformer.

Shannon's Approximations: No Context

0th order: uniform random characters

SAMPLE

XFOML RXKHRJFFJUJ ZLPWCFWKCYJ FFJEYVKCQSGHYD

```
alphabet = "ABCDEFGHIJKLMNOPQRSTUVWXYZ "  
text = ""  
for t in range(T):  
    text += uniform_sample(alphabet)
```

1st order: English letter frequencies

SAMPLE

OCRO HLI RGWR NMIELWIS EU LL NBNESEBYA TH EEI ALHENHTTPA

```
p = count_chars(corpus)  
p = normalize(p)  
text = ""  
for t in range(T):  
    text += sample_from(p)
```

Shannon's Approximations: Adding Context

2nd order: character bigrams

SAMPLE

ON IE ANTSOUTINYS ARE T INCTORE ST BE S DEAMY ACHIN D
ILONASIVE TUC00WE

```
table = count_pairs(corpus)
p_next = normalize_rows(table)
text = "T"
for t in range(T):
    prev = text[-1]
    text += sample_from(p_next[prev])
```

Word-level bigrams

SAMPLE

THE HEAD AND IN FRONTAL ATTACK ON AN ENGLISH WRITER THAT THE
CHARACTER OF THIS POINT IS THEREFORE ANOTHER METHOD

```
table = count_word_pairs(corpus)
p_next = normalize_rows(table)
words = ["THE"]
for t in range(T):
    prev = words[-1]
    words.append(sample_from(p_next[prev]))
```

Examples adapted from Shannon 1948, Section 3. More context produces more coherent samples, but the count table grows quickly.

How Predictable Is English?

Prediction and Entropy of Printed English

C. E. Shannon, Bell System Technical Journal, 1951.

Shannon estimated predictability by asking a human to guess English text one character at a time.

- First-try guesses mean low surprise.
- Many guesses mean high information.
- Better prediction means fewer bits needed to encode the text.
- A good predictor gives an empirical upper bound on entropy.

0.6-1.3 bits/char

Shannon's reported lower and upper bounds for printed English.

$\log_2(27) = 4.75$ bits/char

Uniform random letters plus space: no language structure.

~1.0-1.2 bits/char

Modern/SOTA neural LMs land in the same regime on character-level benchmarks, though the exact number is dataset-dependent.

This is the language-modeling view of compression: if a model predicts text well, it assigns the observed text high probability.

Shannon 1951; Takahashi and Tanaka-Ishii 2018 estimate 1.12 bits/char by neural-LM extrapolation; Radford et al. 2019 report GPT-2 at 0.98 BPC on text8.

From Shannon to the ML Objective

Model the next-symbol distribution

Shannon's experiment asks for the next character. A neural language model asks for the next token.

PREFIX

The model predicts

predicts

TARGET

tokens

$$p(x_{1:T}) = \prod_{t=1}^T p(x_t \mid x_{<t})$$

If we can estimate every conditional, we have a full generative model of text.

Train by expected surprise

The ideal objective is expected negative log-likelihood under the data distribution.

$$\mathcal{L}(\theta) = \mathbb{E}_{x \sim p_{\text{data}}} \left[- \sum_t \log p_{\theta}(x_t \mid x_{<t}) \right]$$

In practice, SGD replaces the expectation with a minibatch average over observed tokens.

This is the likelihood-based version of the generative modeling story from VAEs, flows, diffusion, and flow matching: learn a distribution that makes real data unsurprising.

Cross-Entropy and Perplexity

Average token loss

For a held-out sequence with N predicted tokens:

$$\ell = -\frac{1}{N} \sum_{i=1}^N \log p_{\theta}(x_i \mid x_{<i})$$

- This is cross-entropy in nats when using natural logs.
- Lower loss means the model assigned more probability to the actual text.
- The best possible model matches the true data conditional distribution.

Perplexity

$$\text{PPL} = \exp(\ell)$$

Perplexity is the effective average number of choices the model is uncertain among.

Setting	Loss	Perplexity
Perfect prediction	0	1
Uniform over V tokens	$\log V$	V
Uniform 27 characters	4.75 bits/char	27
Shannon English estimate	0.6-1.3 bits/char	about 1.5-2.5 chars
Token LM benchmark	reported per token	depends on tokenizer and data

PPL = 1 means no uncertainty on the observed sequence. PPL = V means the model is no better than a uniform guess over V choices. Compare perplexities only when tokenizer, data, and protocol are fixed.

The Challenge: Long-Range Dependencies

The **cat** that sat on the mat, which was bought from the store that opened last week near the park where children often play, _____

N-grams

Fixed local window. Long contexts require exponentially large count tables.

RNNs

Entire history is compressed through one evolving hidden state.

Attention

Each token can retrieve directly from previous positions in the context.

Tokenization: BPE Represents Text

From string to token ids

TEXT	unbelievable transformers
START	u n b e l i e v a b l e
MERGE	un believ able transform ers
IDS	1842 9137 481 25918 388

Models do not read strings

The input to the transformer is a sequence of integer token ids.

BPE learns useful chunks

Frequent words stay whole; rare words are represented as reusable subword pieces.

The compromise

Shorter than characters, more flexible than a fixed word vocabulary.

Exact pieces and ids depend on the tokenizer. After BPE, language modeling is ordinary classification: predict the next token id in the vocabulary.

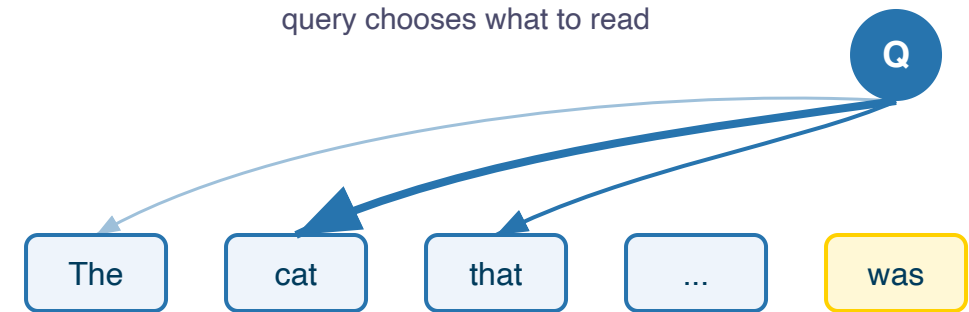
What Attention Changed

Before: compress the past



RNNs can model sequences, but all information must flow through a hidden-state bottleneck.

After: retrieve from context



we want the model to predict this
Training example: I saw a **cat** on a mat

Model prediction: $p(* \mid \text{I saw a})$

cat		0.72
dog		0.10
mat		0.08

Target: cat

Loss: $-\log p(\text{cat})$

The Attention Mechanism

Queries, Keys, and Values

QUERY

What am I looking for?

The current token produces a query vector that asks for relevant context.

KEY

What information do I contain?

Each previous token produces a key vector. Similar query-key pairs get high attention scores.

VALUE

What should I pass forward?

Values are the content vectors that get mixed together after attention weights are computed.

$$\text{Attention}(Q, K, V) = \text{softmax} \left(\frac{QK^\top}{\sqrt{d_k}} \right) V$$

Causal Self-Attention

	The	cat	sat	down
The	1.0	x	x	x
cat	.4	.6	x	x
sat	.1	.7	.2	x
down	.1	.2	.6	.1

The mask enforces generation order

- Each token can attend to itself and earlier tokens.
- Future tokens are hidden during training.
- This lets all positions train in parallel while preserving autoregressive generation.

Project 3: this is the triangular causal mask inside self-attention.

Multi-Head Attention

Head 1

May track local syntax or recent tokens.

the cat sat

Head 2

May track long-range references or subjects.

cat ... was

Head 3

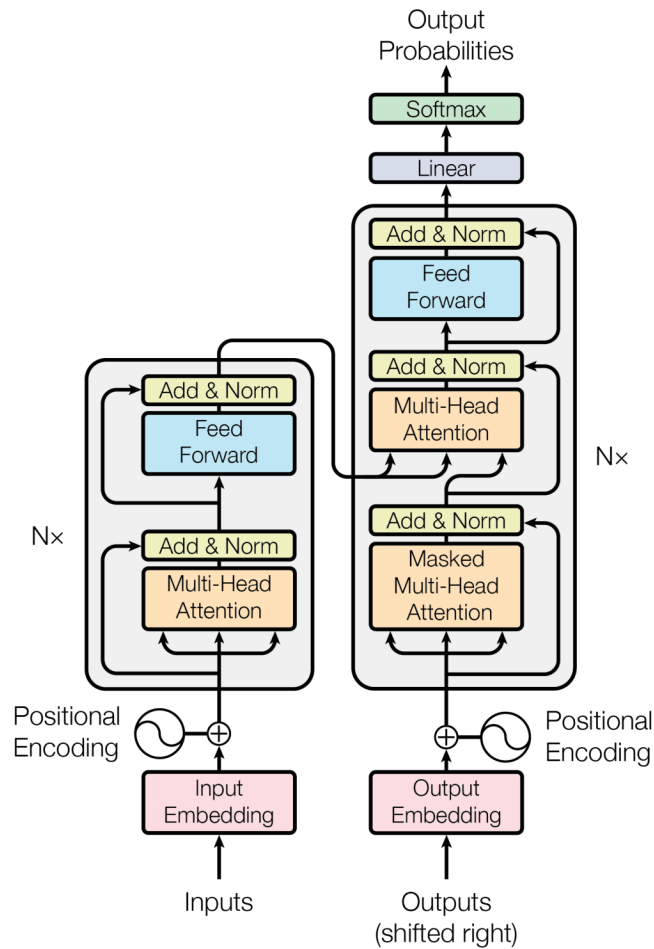
May track positional or delimiter structure.

{ } ;

Multi-head attention gives the model several learned retrieval patterns at each layer.

The Transformer Block

Why Attention Is Not the Whole Block



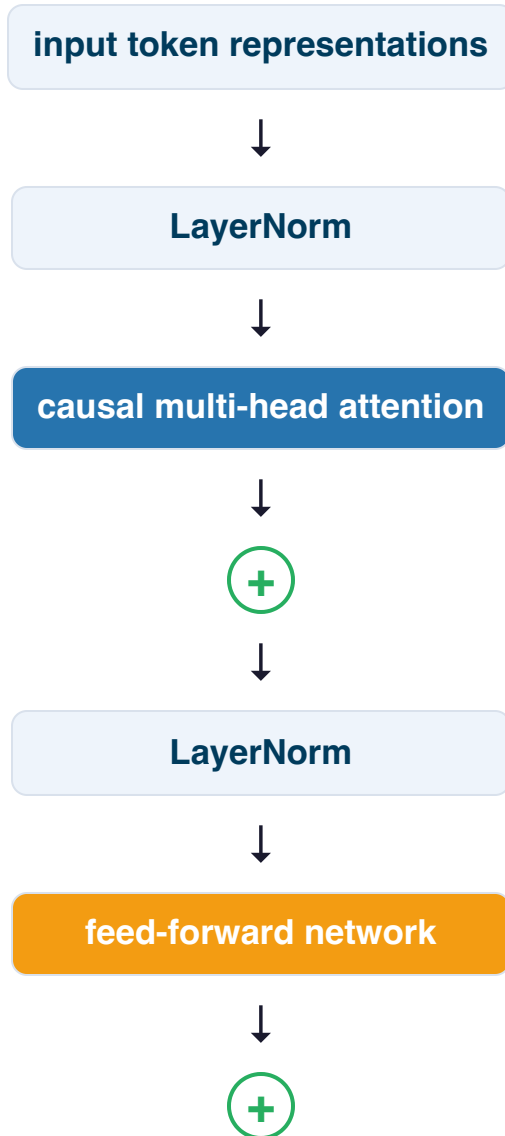
The original paper figure is encoder-decoder

- The **right stack** is the decoder: masked self-attention, cross-attention, FFN.
- GPT removes the encoder and cross-attention, leaving masked self-attention + FFN blocks.
- Residuals and normalization are not decoration; they make deep stacks trainable.

Project 3 uses the decoder-only version of the right stack.

Figure source: Vaswani et al., "Attention Is All You Need," arXiv:1706.03762 source, Figure 1.

The Decoder Block



Pre-norm decoder block

$$h = x + \text{MHA}(\text{LN}(x))$$

$$y = h + \text{FFN}(\text{LN}(h))$$

- Masking makes it decoder-only.
- Residuals preserve an information highway.
- Stacking these blocks gives GPT.

Embedding Table: Tokens to Vectors

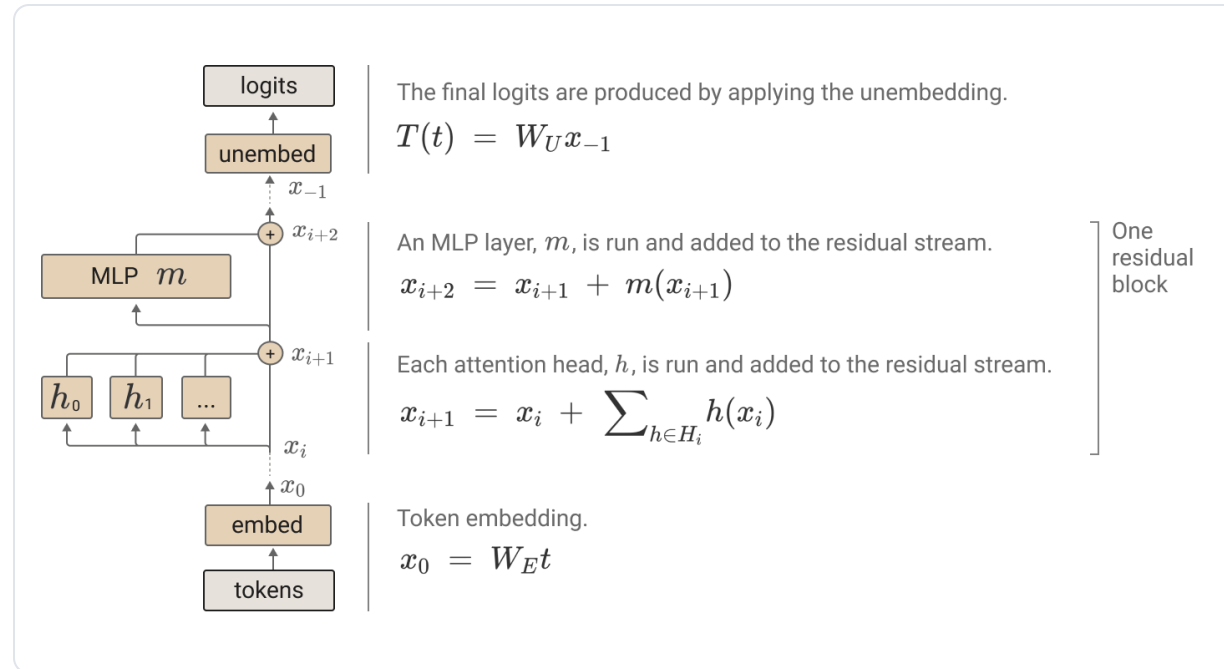


Figure source: Elhage et al., "A Mathematical Framework for Transformer Circuits," Anthropic, 2021.

The embedding table starts the residual stream

$$x_0 = W_E t$$

- A token id is a row lookup in a learned table.
- At this point the representation has no context from other tokens.
- Position information is added so identical tokens at different positions can behave differently.

The model never sees raw strings after tokenization; it sees vectors in the residual stream.

Attention: Mix Across Tokens

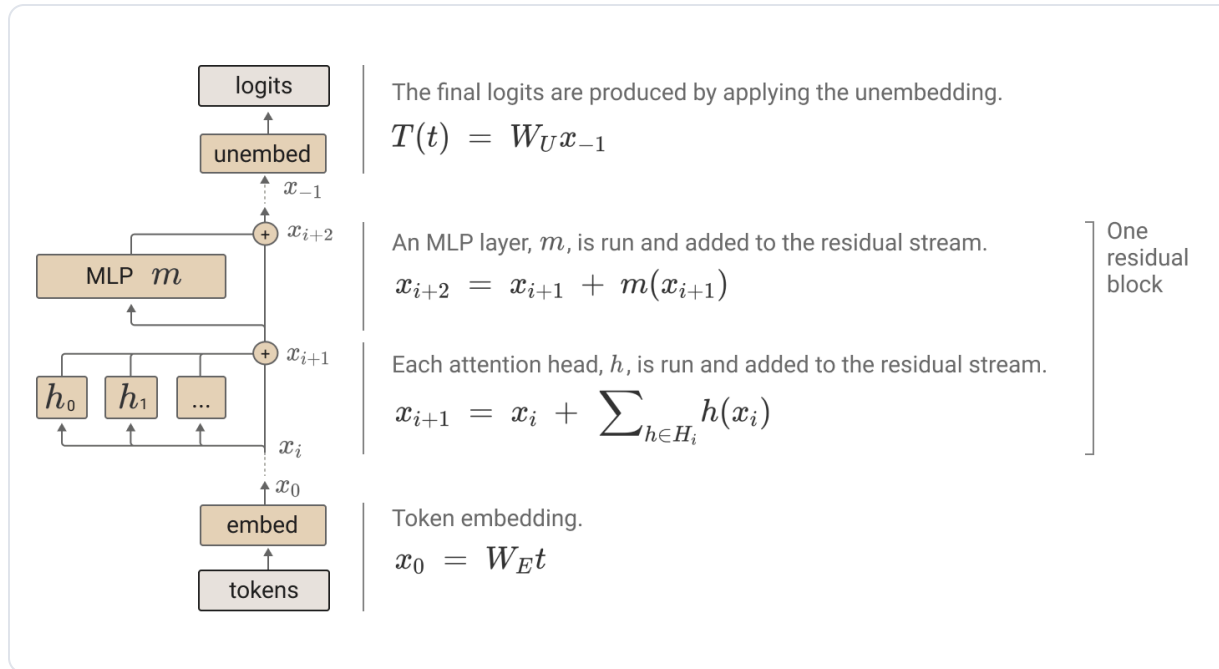


Figure source: Elhage et al., Anthropic, 2021.

Attention writes contextual updates

$$x_{i+1} = x_i + \sum_{h \in H_i} h(x_i)$$

- Queries and keys decide which previous positions matter.
- Values carry the information that gets moved.
- The output is added back into each token's residual stream.

This is the first place token representations become contextual.

Feed-Forward: Lift and Project Back

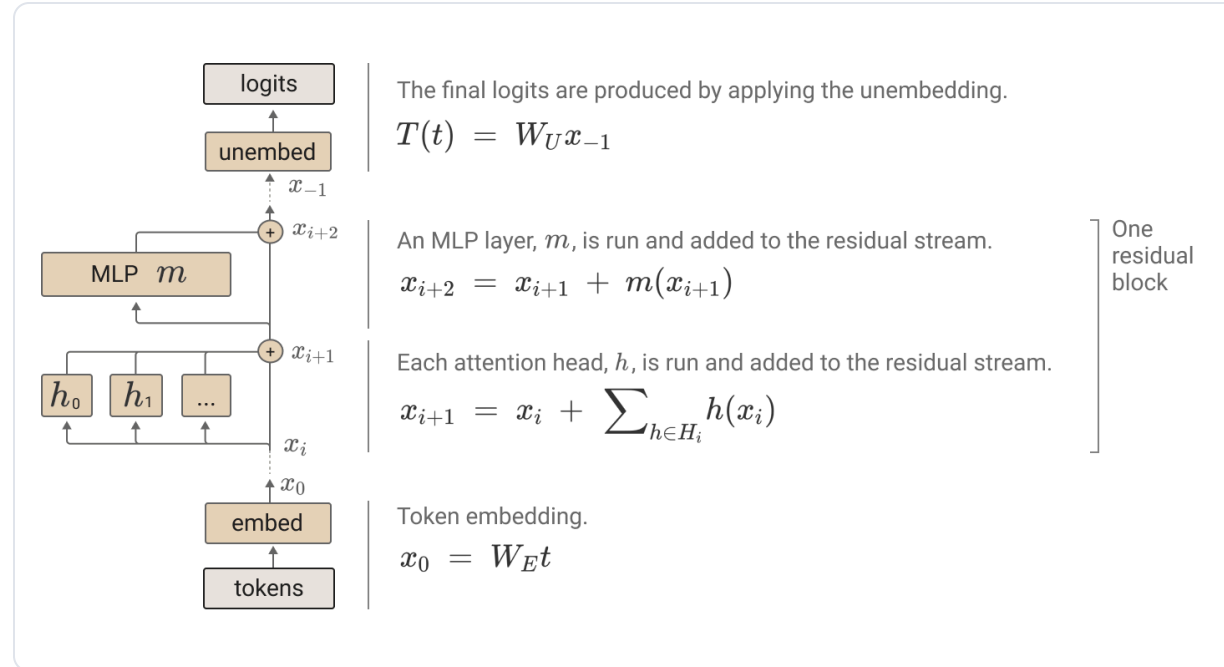


Figure source: Elhage et al., Anthropic, 2021.

The MLP transforms each token independently

$$m(x) = W_{\text{out}} \sigma(W_{\text{in}} x)$$

- It expands into a wider hidden dimension.
- The nonlinearity turns features on and off.
- It projects back to the residual-stream width and adds the result.

Attention moves information between tokens; the feed-forward network processes the information at each token.

Sampling: From Logits to Text

One generation step

$$p(x_{t+1} \mid x_{\leq t}) = \text{softmax}(W_U h_t)$$

- 1 Run the transformer on the current context.
- 2 Use the unembedding matrix to produce logits over the vocabulary.
- 3 Softmax turns logits into next-token probabilities.
- 4 Choose one token, append it, and repeat.

Sampling choices

- **Greedy:** always choose the most likely token.
- **Temperature:** sharpen or flatten the distribution before sampling.
- **Top-k / top-p:** sample only from a restricted high-probability set.

The model predicts tokens

Training scores the observed next token. Generation samples from the same distribution.

GPT and the Scaling Moment

GPT: Decoder-Only Pretraining

token ids

token + position
embeddings

N decoder blocks

linear LM head

next-token
distribution

$$\mathcal{L} = - \sum_t \log p_{\theta}(x_t \mid x_{<t})$$

GPT is not a new objective. It is the next-token objective paired with a scalable architecture and a lot of data.

GPT -> GPT-2 -> GPT-3

2018

GPT

- Decoder-only transformer pretrained with next-token prediction.
- Pretrain first, adapt later: the recipe becomes recognizable.
- The important conceptual move is not a new loss; it is scale-friendly pretraining.

117M
params

Books
pretrain

NTP
loss

Radford et al., 2018.

2019

GPT-2

- Same core architecture and objective, trained larger.
- Generated coherent passages and showed zero-shot task behavior.
- Prompts like **TL;DR:** could steer the model into summarization.
- Framed language modeling as unsupervised multitask learning.

1.5B
params

WebText
data

zero-shot
tasks

Radford et al., 2019.

2020

GPT-3

- 175B parameters and much larger training compute.
- Few-shot prompting becomes a central capability.
- Turns scale into the question: how far does this recipe go?

175B
params

~300B
tokens

few-shot
prompts

Brown et al., 2020.

GPT-3: In-Context Learning

The three settings we explore for in-context learning

Zero-shot

The model predicts the answer given only a natural language description of the task. No gradient updates are performed.

```
1 Translate English to French: ← task description
2 cheese => ..... ← prompt
```

One-shot

In addition to the task description, the model sees a single example of the task. No gradient updates are performed.

```
1 Translate English to French: ← task description
2 sea otter => loutre de mer ← example
3 cheese => ..... ← prompt
```

Few-shot

In addition to the task description, the model sees a few examples of the task. No gradient updates are performed.

```
1 Translate English to French: ← task description
2 sea otter => loutre de mer ← examples
3 peppermint => menthe poivrée ←
4 plush giraffe => girafe peluche ←
5 cheese => ..... ← prompt
```

Traditional fine-tuning (not used for GPT-3)

Fine-tuning

The model is trained via repeated gradient updates using a large corpus of example tasks.

```
1 sea otter => loutre de mer ← example #1
↓
gradient update
↓
1 peppermint => menthe poivrée ← example #2
↓
gradient update
↓
...
↓
1 plush giraffe => girafe peluche ← example #N
↓
gradient update
↓
1 cheese => ..... ← prompt
```

The surprising part

- The model is not updated for a new task.
- The task is described inside the context window.
- Examples in the prompt can change the behavior at inference time.

GPT-3 made prompting feel like an interface to a learned task distribution.

Figure source: Brown et al., "Language Models are Few-Shot Learners," arXiv:2005.14165 source, Figure 2.1.

Scaling Laws: The GPT-3 Question

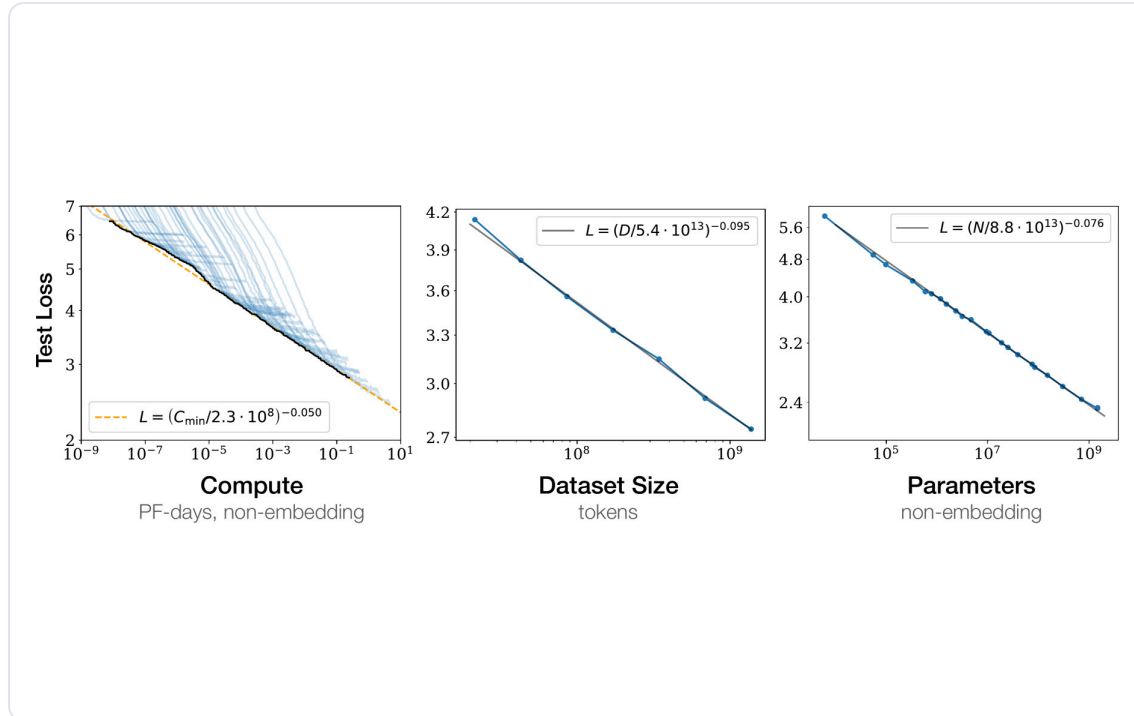


Figure source: Kaplan et al., "Scaling Laws for Neural Language Models," arXiv:2001.08361 source, Figure 1.

The empirical punchline

- Loss decreases predictably as models, data, and compute scale.
- The same architecture/objective can keep getting better.
- The GPT-3-era lesson was: if loss follows a power law, scale is a reliable lever.

This explains why GPT-3 felt like a phase change: the field had an empirical reason to believe the recipe would keep improving.

Kaplan et al. 2020; Hoffmann et al. 2022.

Kaplan Takeaway: Scale the Whole Recipe

The first scaling-law message was not "find a clever new loss." It was: this simple recipe improves predictably when we scale the axes together.

More parameters

Larger models can represent more structure, but only if they see enough data to use the capacity.

N
parameters

More tokens

More data gives more learning signal, but only if the model has enough capacity and compute.

D
training tokens

More compute

Compute is the budget that lets us increase model size and data instead of trading them off blindly.

C
training FLOPs

GPT-3-era lesson

Scaling the decoder-only next-token recipe works: larger models trained on more text keep improving smoothly.

The Chinchilla question

Given a fixed compute budget, how much should go to parameters versus tokens?

$$C \approx 6ND$$

Chinchilla: Read the IsoFLOP Plot

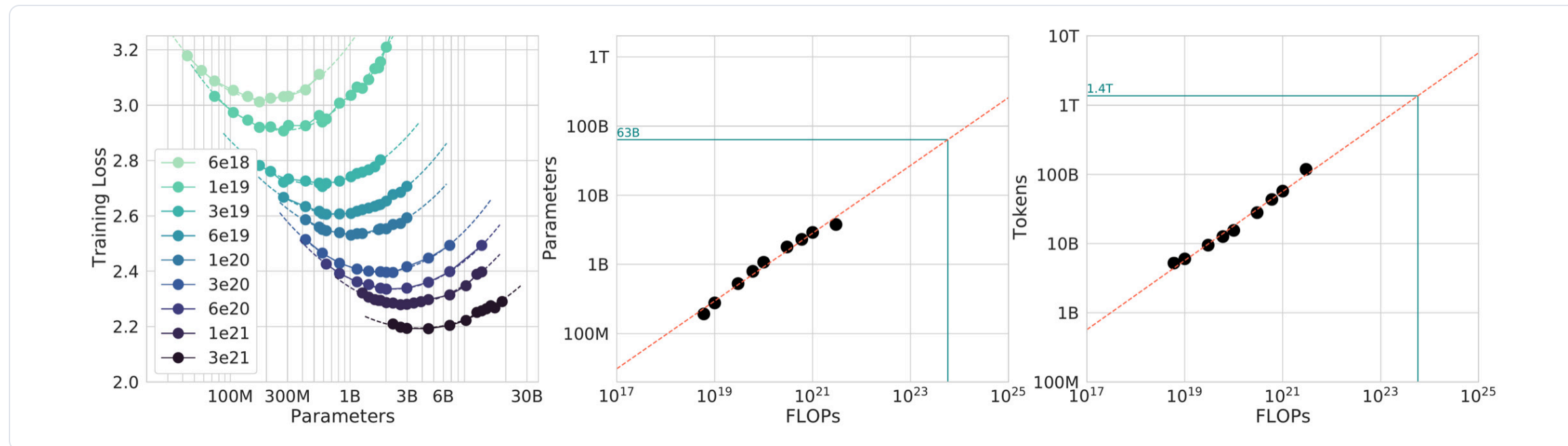


Figure source: Hoffmann et al., "Training Compute-Optimal Large Language Models," arXiv:2203.15556 source, IsoFLOP curves figure.

1. Fix compute

Each curve holds the FLOP budget constant.

2. Sweep parameters

Changing model size also changes how many tokens fit into that budget.

3. Find the valley

The minimum loss gives the compute-optimal model size and token count.

Rule of thumb: $C \approx 6ND$, so compute-optimal Chinchilla uses $D/N \approx 20$ tokens per parameter: 70B params -> 1.4T tokens.

The Pretraining Stack

The pretraining stack

- **Training algorithm/loss:** next-token cross-entropy
- **Architecture:** decoder-only transformer
- **Data:** text tokens at scale
- **Evaluation:** perplexity, downstream tasks, generation quality
- **Systems:** parallel training and efficient inference

What the story adds up to

We now have the loss, the architecture, and the scaling picture: useful pretraining is a joint choice of model size, data, and compute.

$$\min_{\theta} - \sum_t \log p_{\theta}(x_t \mid x_{<t})$$

GPT-2/3 made scale the central lever. Chinchilla made the data allocation part of the recipe.

What we postpone

- Instruction tuning and chat formatting
- RLHF, DPO, reward models
- RAG, tools, agents
- Reasoning RL and test-time compute

The next lectures explain how pretrained models become assistants and reasoning systems.

Example: What Does “Data” Mean?

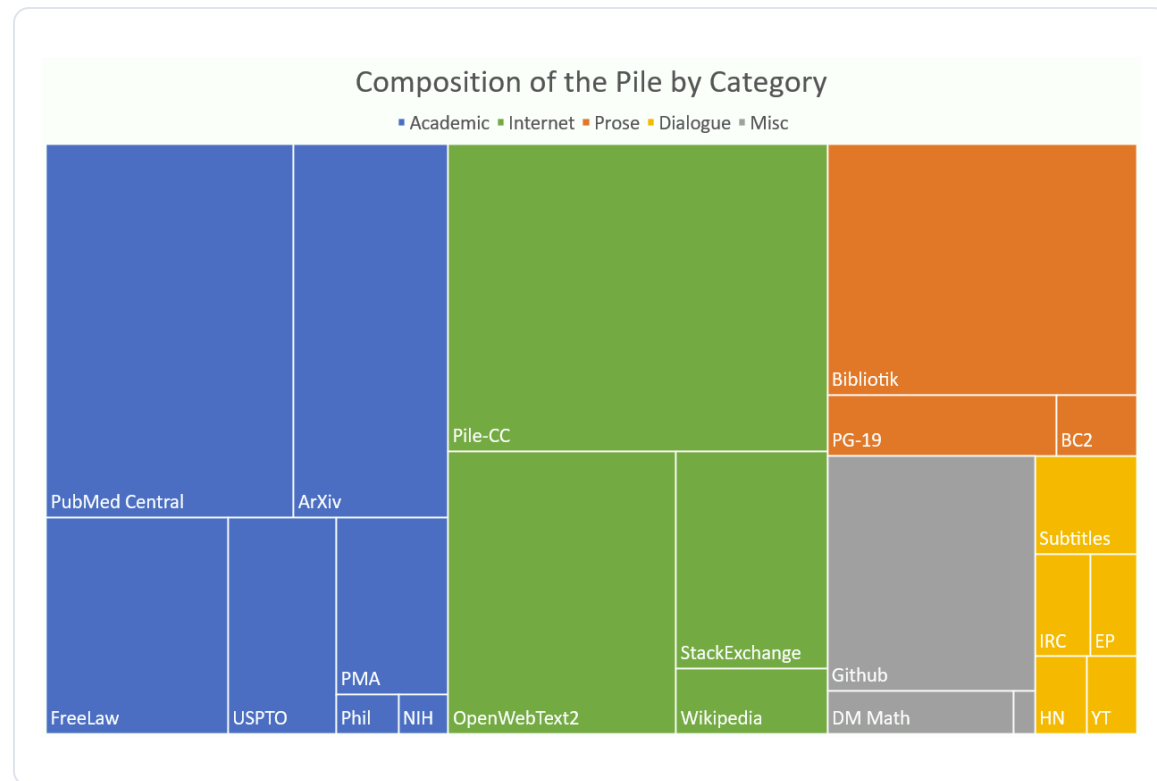


Figure source: Gao et al., "The Pile: An 800GB Dataset of Diverse Text for Language Modeling," arXiv:2101.00027 source, Figure 1.

The Pile as a concrete example

- 825 GiB English corpus built from 22 components.
- Mixture includes filtered web text, books, academic papers, code, law, forums, subtitles, and reference text.
- Components are sampled with different epoch weights; the figure shows effective size.
- The data mixture is a modeling choice: it changes what domains the model sees often enough to learn well.

Chinchilla tells us data allocation matters. The Pile is here to show what "text at scale" looks like in real pretraining.

Project 3 Mirrors the Pretraining Stack

Bigram baseline

Predict the next token from a tiny local context. This is the Shannon-style idea in code.

```
BigramLanguageModel
```

TinyStories

A constrained text world where small models can still learn recognizable narrative patterns.

GPT-2 tokenizer, context length, generation

MiniGPT

Use embeddings, learned positions, causal self-attention, FFNs, LayerNorm, residuals, and a language-modeling head.

```
MiniGPT -> generate()
```

The project is the same recipe at a size where we can inspect every part.

Optional: From Completion Models to Chat

Before chat: complete the text

...

PROMPT

Write a short explanation of attention in transformers:

Attention lets each token compare itself to earlier tokens and copy useful information into its representation. This makes it easier to model long-range dependencies...

The base model has one native behavior: continue the token stream.



post-training
changes the
interface and
preferred
behavior

After chat: follow the user

...

User: Explain attention to a student who knows matrix multiplication.

Assistant: Think of each token making a query, comparing it to keys from previous tokens, and averaging the corresponding values.

Instruction tuning and preference tuning teach the model which continuations are helpful, harmless, and conversational.

The objective is still next-token prediction. The major change is the data distribution: demonstrations, comparisons, chat formatting, and feedback.

Optional: What Do We Evaluate?

There is no single number for "model quality." Each benchmark measures a proxy for some behavior we care about.

Language modeling

Held-out loss and perplexity measure how much probability the model assigns to real text.

Good for: pretraining progress.

Task accuracy

Benchmarks such as MMLU or BIG-bench turn model behavior into scored answers.

Good for: broad capability snapshots.

Generation quality

Human preference or model-judge comparisons ask which output is more useful.

Good for: chat and assistant behavior.

Reliability

Factuality, calibration, robustness, and safety evaluations stress-test failure modes.

Good for: deployment risk.

Example benchmark families: MMLU, BIG-bench, HELM, MT-Bench, Chatbot Arena. Exact comparisons depend on prompt format, decoding, data, and evaluation protocol.

Optional: How to Read Benchmark Tables

The evaluation pipeline

- 1 Choose a dataset and prompt format.
- 2 Run the model with a decoding rule.
- 3 Score with an exact answer, rubric, human judge, or model judge.
- 4 Aggregate scores and compare systems.

Questions to ask before trusting the number

Issue	Why it matters
Prompting	Zero-shot, few-shot, chain-of-thought, and formatting can change scores.
Contamination	Was the test set seen during pretraining?
Saturation	Easy benchmarks stop separating strong models.
Judging	Human and model judges have biases and variance.
Distribution shift	A benchmark may not match real use.

For Project 3, the same principle applies: report loss/perplexity, inspect samples, and ask what each tells you that the other does not.

What We Covered Today

Generative modeling of text as autoregressive next-token prediction.

Why context is the central difficulty.

How attention performs learned retrieval over the context.

How decoder blocks become GPT-style language models.

Why GPT-2/GPT-3, scaling laws, and Chinchilla changed the trajectory of pretraining.

Next lecture: scale works, but base models are not assistants. We turn pretrained models into useful systems.

Lecture 2 Start: Post-Training

ChatGPT: The Behavior Shift

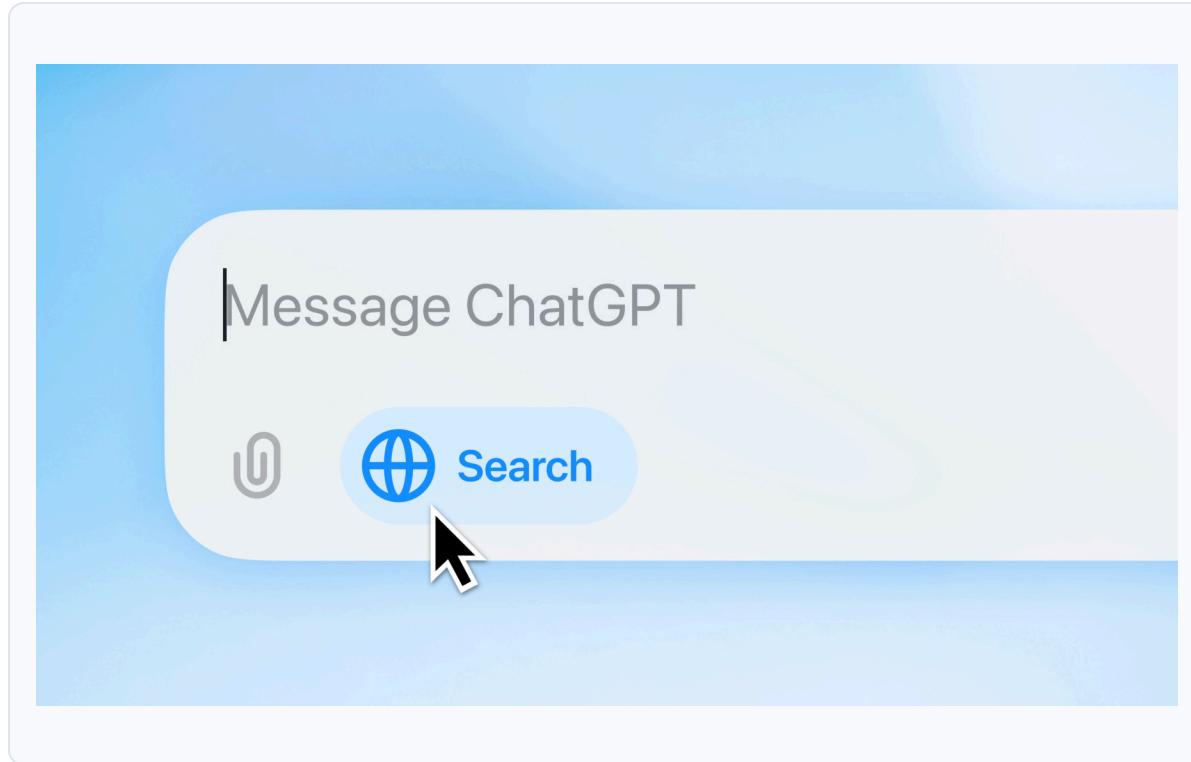


Figure source: Nathan Lambert, SALA 2026 RLHF/post-training deck, ChatGPT slide.

Same foundation, different behavior

- Base models continue text.
- Chat models follow instructions in a turn-taking interface.
- The behavior shift comes from post-training data and optimization.

ChatGPT did not make next-token prediction irrelevant. It changed which next tokens were rewarded.

Question 1: What Changed at ChatGPT?

Base model behavior

Prompt: The president of the United States in 2006 was

Base completion: George W. Bush, the governor of Florida in 2006 was Jeb Bush, and John McCain was an Arizona senator...

The model continues a plausible text stream.

Assistant behavior

User: The president of the United States in 2006 was

Assistant: George W. Bush was president in 2006. He served from January 20, 2001 to January 20, 2009.

The model answers the implied user request.

The historical shift is from completion quality to interaction quality. That requires new data and new optimization targets.

Lecture 2 Guiding Questions

1. What behavior do we want?

Pretraining learns text. Assistants need instruction following, calibrated refusal, formatting, and multi-turn behavior.

2. How do we get labels?

Writing ideal answers is expensive. Comparing two answers is often easier, faster, and more robust.

3. What do we optimize?

SFT optimizes demonstrations. Reward modeling fits preferences. PPO/DPO optimize policies against those preferences.

4. What can go wrong?

Reward hacking, over-optimization, labeler bias, benchmark gaming, and distribution shift.

5. Why not just use RL?

Language rewards are sparse, learned, noisy, and response-level. Stability matters.

6. What came after RLHF?

DPO, RLVR, synthetic data, model judges, rejection sampling, and reasoning-specific training.

Why Post-Training Exists

Pretraining objective

$$-\log p_{\theta}(x_t \mid x_{<t})$$

Excellent for learning language, facts, code, and patterns from internet-scale text.

Assistant objective

- Answer the user directly.
- Use the requested format.
- Avoid unsafe or misleading behavior.
- Prefer helpful completions over unhelpful ones.

Many behaviors are easy for people to judge but hard to write as an exact differentiable reward. RLHF turns judgments into an optimization signal.

Why Human Feedback?

Some objectives are easy to judge but hard to specify

- Which answer is more helpful?
- Which summary is more faithful?
- Which refusal is appropriate?
- Which reasoning trace reaches the right conclusion?

RLHF starts from this asymmetry: people can often compare behavior before they can write a reward function for it.

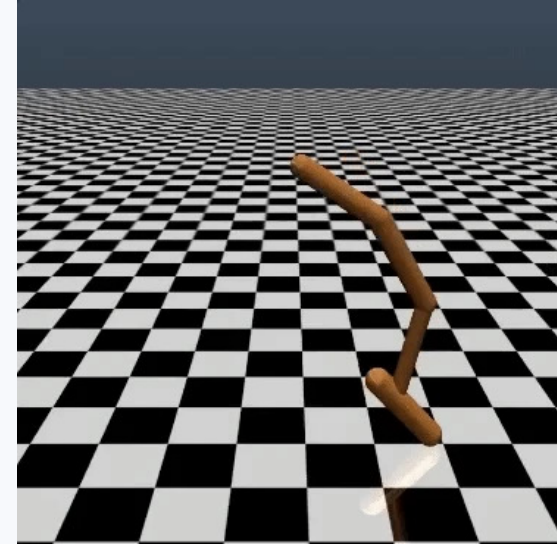
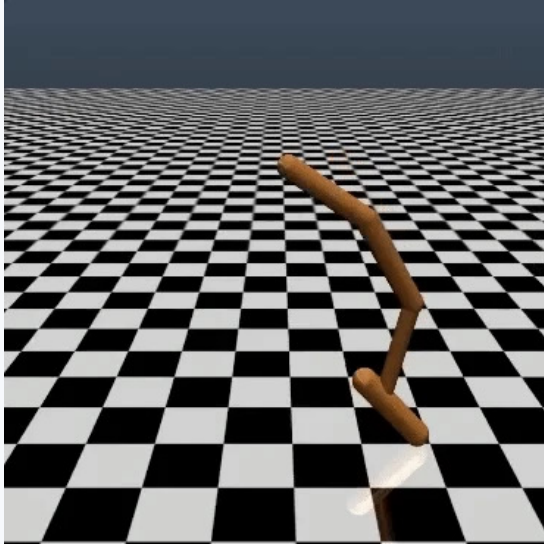
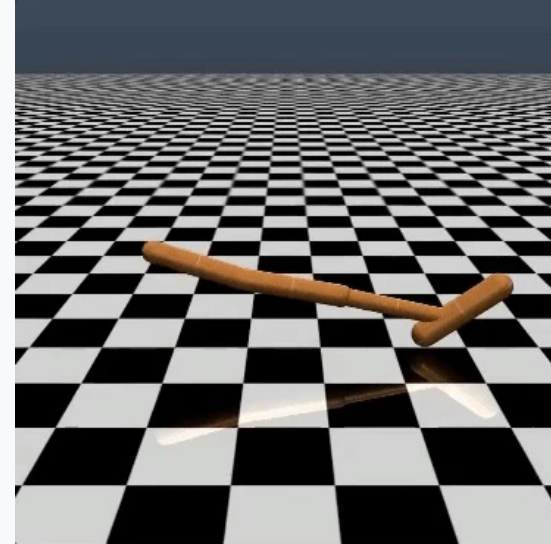


Figure source: Lambert SALA 2026 deck, based on Christiano et al. 2017.

Human Feedback vs. Hand-Written Rewards



Human judges compare behavior.



Hand-written rewards can miss what humans actually want.

RLHF says: learn a reward model from judgments, then optimize the policy against that learned model.

RLHF Before Language Models

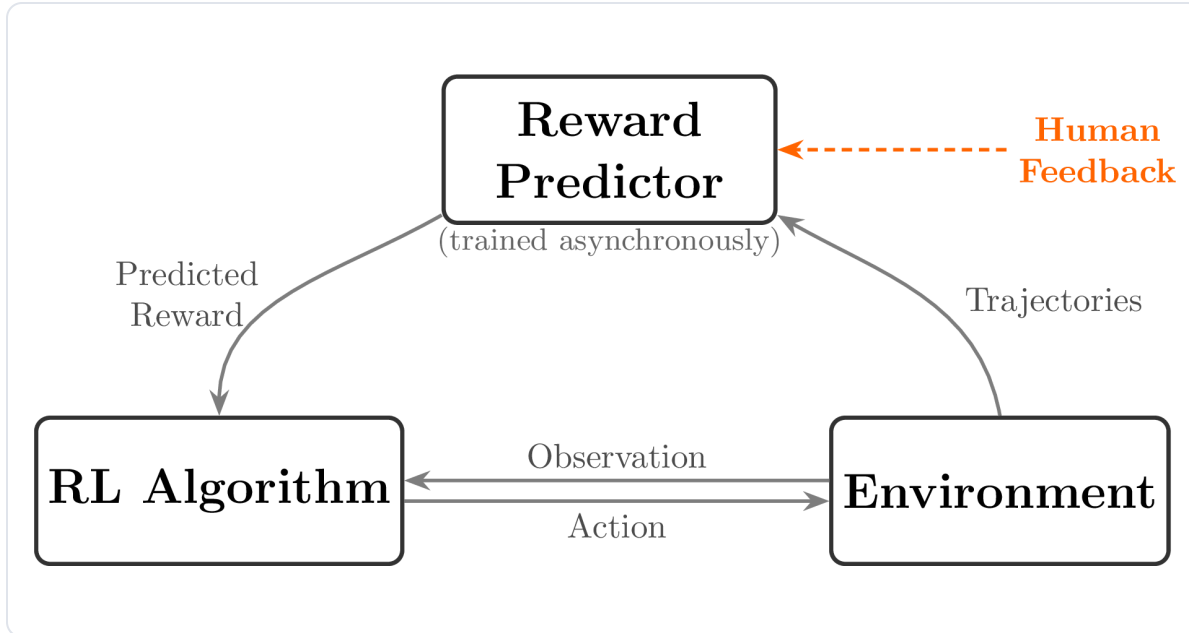


Figure source: Lambert SALA 2026 deck, based on Christiano et al. 2017.

Historical idea

- **TAMER**: learn from human feedback on actions.
- **Christiano et al. 2017**: learn rewards from trajectory preferences.
- **Ziegler et al. 2019**: apply preference learning to language models.
- **InstructGPT 2022**: make the three-stage language-model recipe canonical.

The Path to Modern RLHF

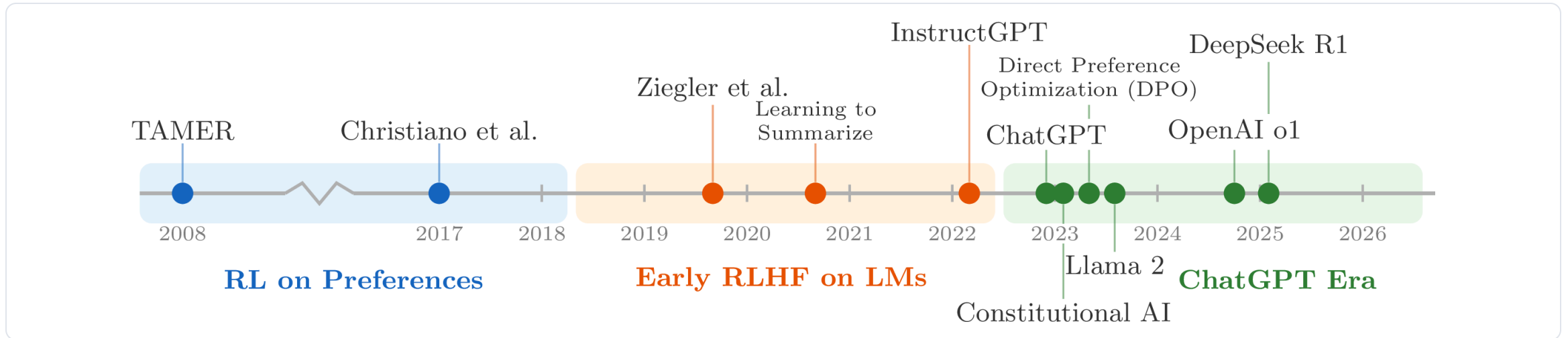


Figure source: Lambert SALA 2026 RLHF/post-training deck.

The story is not one algorithm replacing another. The recurring pattern is: collect better behavioral data, choose an optimizer, and control how far the model moves.

InstructGPT: The Canonical RLHF Recipe

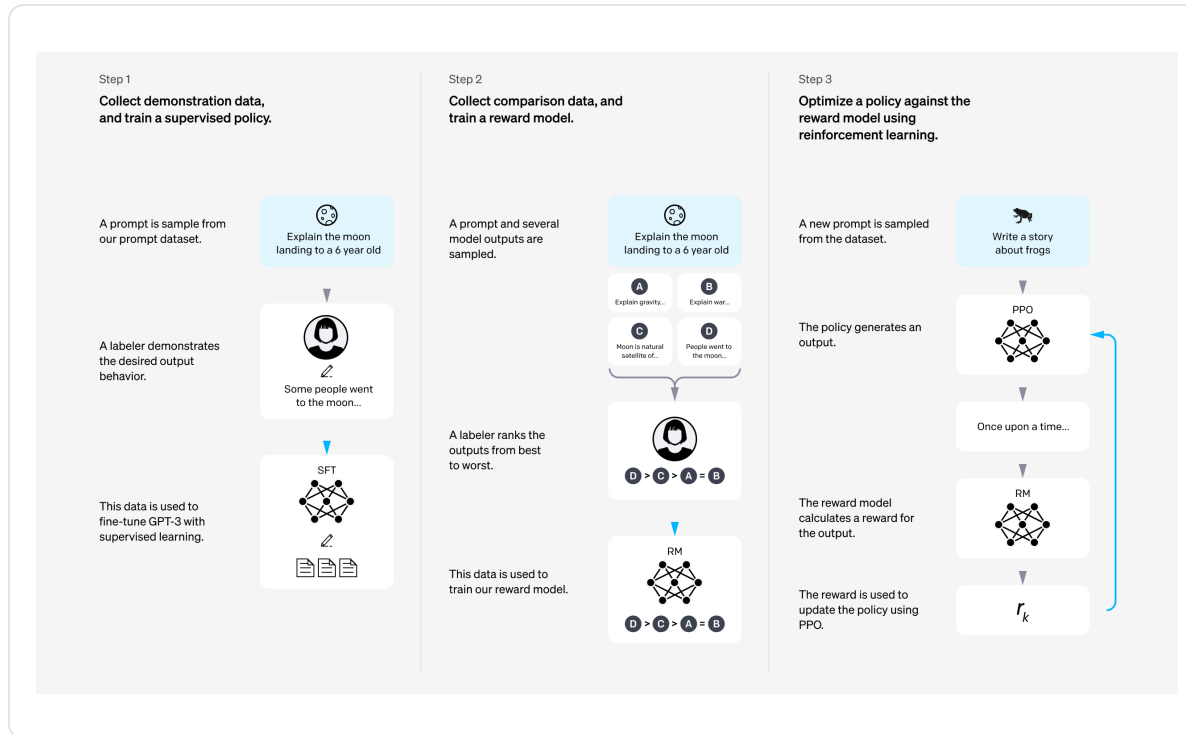


Figure source: Nathan Lambert SALA 2026 deck, adapted from Ouyang et al. 2022.

1. Supervised fine-tuning

Train on demonstrations of desired assistant behavior.

2. Reward modeling

Train a model to predict which response humans prefer.

3. RL against the reward model

Optimize the assistant to get high reward while staying close to the reference model.

Step 1: Supervised Fine-Tuning

Train on demonstrations

Start from the pretrained model and fine-tune on prompt-response pairs written or curated to look like good assistant behavior.

$$\mathcal{L}_{\text{SFT}}(\theta) = - \sum_{(x, y^*)} \sum_t \log \pi_{\theta}(y_t^* \mid x, y_{<t}^*)$$

Example training row

User: Write a short poem about an optimistic goldfish.

Assistant: Bright little goldfish
Finds a sunrise in each wave
Small bowl, endless hope

SFT teaches the format. It does not by itself solve preference tradeoffs.

Step 2: Preference Data

Instead of asking annotators to write the perfect answer, ask which of two answers is better. This is often easier and more scalable.

chosen

Response A

Attention lets each token decide which earlier tokens to read from. It uses query-key similarity to compute weights, then averages value vectors and writes that information back into the residual stream.

Response B

Attention is when the model pays attention to words. It helps the model understand the sentence better and makes answers more accurate.

$(x, y_w, y_l) = \text{prompt, chosen response, rejected response}$

Preference Data: Rankings Become Pairs

Annotator task

For the same prompt, sample multiple candidate responses and ask labelers to rank or compare them.

Prompt x $\rightarrow$ responses $y_1, y_2, y_3, y_4 \rightarrow$ preference order

Rankings can be converted into many pairwise chosen/rejected examples.

Why pairs are useful

- They avoid requiring an absolute quality score.
- They let us train with logistic likelihood.
- They scale across subjective qualities like helpfulness and style.
- They expose tradeoffs: concise versus complete, safe versus useful.

$$(y_1 \succ y_3), (y_1 \succ y_4), (y_2 \succ y_3), \dots$$

Reward Model: A Scalar Head on a Language Model

Input

Concatenate the prompt and response, then run them through a transformer.

[prompt tokens] [response tokens]

Output

Replace the next-token head with a scalar reward head.

$$r_{\phi}(x, y) \in \mathbb{R}$$

The scalar is not "truth." It is a learned prediction of preference.

This is why Bradley-Terry matters: it tells us how to train scalar scores from pairwise human judgments.

Bradley-Terry Reward Model

Turn preferences into probabilities

The reward model assigns a scalar score to each prompt-response pair.

$$r_{\phi}(x, y) \in \mathbb{R}$$

A response should win when it receives a higher reward score.

Bradley-Terry likelihood

PREFERENCE PROBABILITY

$$P(y_w \succ y_l \mid x) = \sigma(r_{\phi}(x, y_w) - r_{\phi}(x, y_l))$$

TRAINING LOSS

$$\mathcal{L}_{\text{RM}}(\phi) = -\log \sigma(r_{\phi}(x, y_w) - r_{\phi}(x, y_l))$$

This is the optimization bridge: human comparisons become a differentiable model of preference.

Reward Models Are Proxy Objectives

They generalize imperfectly

The reward model only sees finite comparison data. Out-of-distribution responses can exploit its blind spots.

They compress many values

Helpfulness, harmlessness, honesty, style, and instruction following are squeezed into one scalar.

They can be over-optimized

If the policy chases reward too aggressively, the response can get worse even while predicted reward improves.

This motivates the PPO/KL machinery: optimize preference, but keep the policy close enough to the model we trust.

Step 3: Optimize Against the Reward Model

Policy optimization loop

- Sample prompts x .
- Generate responses $y \sim \pi_{\theta}(\cdot | x)$.
- Score responses with $r_{\phi}(x, y)$.
- Update the policy, usually with a KL penalty to stay near a reference model.

$$J(\pi) = \mathbb{E}[r_{\phi}(x, y)] - \beta D_{\text{KL}}(\pi || \pi_{\text{ref}})$$

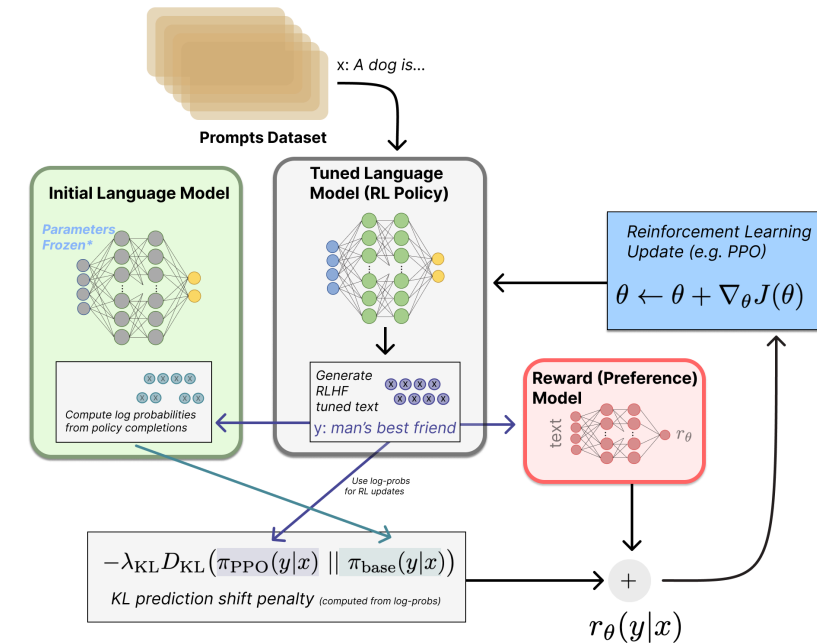


Figure source: Nathan Lambert SALA 2026 deck, RLHF overview slide.

The KL Penalty Is the Guardrail

Why not maximize reward directly?

- The reward model is an imperfect proxy.
- Optimization can exploit proxy errors.
- Large policy shifts can degrade language quality or safety.

Reference-model anchoring

$$D_{\text{KL}}(\pi_{\theta}(\cdot \mid x) \parallel \pi_{\text{ref}}(\cdot \mid x))$$

The reference model keeps the optimized assistant close to the SFT model unless the reward improvement is worth the move.

RLHF is not just "make reward high." It is constrained optimization under a learned, noisy preference model.

PPO in RLHF: What Gets Updated?

One response is a trajectory

The prompt is fixed, the policy samples a whole response, and the reward usually arrives at the end.

$$y = (y_1, \dots, y_T), \quad \pi_{\theta}(y \mid x) = \prod_t \pi_{\theta}(y_t \mid x, y_{<t})$$

RLHF is often closer to contextual bandit optimization than to long-horizon robotics RL.

PPO uses an old policy as a trust region

PROBABILITY RATIO

$$\rho_t(\theta) = \frac{\pi_{\theta}(y_t \mid h_t)}{\pi_{\text{old}}(y_t \mid h_t)}$$

CLIPPED OBJECTIVE

$$\mathcal{L}_{\text{clip}} = \mathbb{E} [\min(\rho_t A_t, \text{clip}(\rho_t, 1 - \epsilon, 1 + \epsilon) A_t)]$$

The clip prevents a single update from making the new policy too different from the policy that generated the samples.

PPO in RLHF: Reward, KL, Advantage

Reward

The learned reward model scores the completed response.

$$r_{\phi}(x, y)$$

KL penalty

Subtract a cost for moving away from the reference model.

$$-\beta \log \frac{\pi_{\theta}(y \mid x)}{\pi_{\text{ref}}(y \mid x)}$$

Advantage

Estimate whether this response was better than expected.

$$A \approx R - V(x)$$

Why PPO was used

- Works with sampled text, not just fixed labels.
- Can optimize a learned scalar reward directly.
- Has update-size controls: clipping and KL.

Why it is delicate

- Expensive: generate, score, and update repeatedly.
- Sensitive to reward-model errors.
- Needs careful KL, reward normalization, and value modeling.

DPO: Eliminate the Explicit Reward Model

Start from KL-regularized RLHF

$$\max_{\pi} \mathbb{E}_{y \sim \pi(\cdot | x)} [r(x, y)] - \beta D_{\text{KL}} (\pi(\cdot | x) \| \pi_{\text{ref}}(\cdot | x))$$

The optimal policy has a closed-form relationship to the reward.

Reward as a log-ratio

$$\pi^*(y | x) = \frac{1}{Z(x)} \pi_{\text{ref}}(y | x) \exp \left(\frac{1}{\beta} r(x, y) \right)$$

$$r(x, y) = \beta \log \frac{\pi^*(y | x)}{\pi_{\text{ref}}(y | x)} + \beta \log Z(x)$$

In a pairwise comparison, the unknown $\log Z(x)$ cancels. This lets us train from preferences without separately fitting r_{ϕ} .

DPO Loss: Bradley-Terry With Policy Ratios

Define an implicit reward score

$$s_{\theta}(x, y) = \beta [\log \pi_{\theta}(y \mid x) - \log \pi_{\text{ref}}(y \mid x)]$$

Chosen responses should have larger implicit reward than rejected responses.

Pairwise objective

$$\mathcal{L}_{\text{DPO}}(\theta) = -\log \sigma(s_{\theta}(x, y_w) - s_{\theta}(x, y_l))$$

- Increase likelihood of chosen completions relative to reference.
- Decrease likelihood of rejected completions relative to reference.
- β controls how strongly we move away from the reference.

DPO is not a new kind of preference data. It is a different optimizer for the same chosen-versus-rejected signal.

PPO vs. DPO

Axis	PPO-style RLHF	DPO-style preference optimization
Training signal	Reward model scores sampled responses.	Chosen/rejected pairs directly.
Generation during training	Online rollouts from the current policy.	Usually offline preference dataset.
Extra models	Reward model, value model, reference model.	Reference model; implicit reward via log-ratios.
Main control knob	KL coefficient, PPO clip, reward normalization.	β against the reference policy.
Failure modes	Reward hacking, instability, high cost.	Data quality limits, overfitting pairwise preferences.

Modern post-training often mixes these ideas: supervised data, preference data, direct objectives, RL on verifiable tasks, and filtering/evaluation loops.

What Came After Classic RLHF?

DPO and direct preference methods

Train directly on chosen/rejected pairs without fitting an explicit reward model first.

RLVR

Use verifiable rewards when the answer can be checked automatically, such as math or code.

Modern post-training

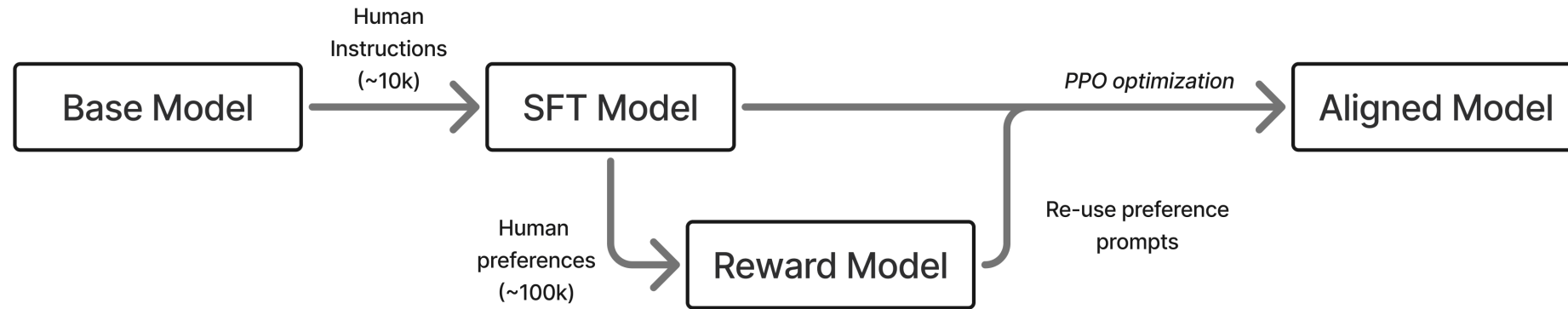
Multiple stages: filtering, SFT, preference optimization, RL, tool data, safety data, and sometimes reasoning-specific training.

DPO objective shape

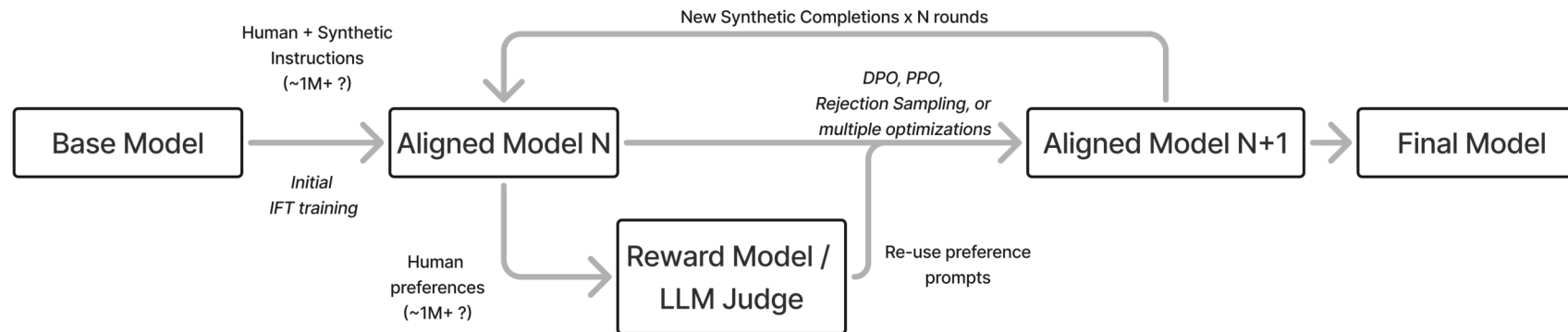
$$\mathcal{L}_{\text{DPO}}(\theta) = -\log \sigma \left(\beta \log \frac{\pi_{\theta}(y_w | x)}{\pi_{\text{ref}}(y_w | x)} - \beta \log \frac{\pi_{\theta}(y_l | x)}{\pi_{\text{ref}}(y_l | x)} \right)$$

The same preference data can train the policy more directly.

From RLHF to Post-Training



Early recipe: supervised fine-tuning, reward modeling, RL.



Modern recipe: many staged optimizers, filters, datasets, and evaluations. Figure source: Nathan Lambert SALA 2026 deck.

Lecture 2 Roadmap

01

ChatGPT

What changed relative to base models?

02

SFT

Use demonstrations to teach the assistant format.

03

RM

Fit Bradley-Terry reward models from comparisons.

04

RLHF/DPO

Optimize preferences while staying anchored.

05

Beyond

RLVR, reasoning models, tools, and agents.

Lecture 1: how base models are pretrained. Lecture 2: how base models become useful assistants. Lecture 3: how assistants become reasoning and agentic systems.